

滴滴前端一面（5年社招）

滴滴前端一面（5年社招）

- 1.react fiber
- 2.http1.1、http2、http3
- 3.https和http的区别，http是怎么加密的
- 4.OSI 七层网络模型
- 5.tcp和udp区别
- 6.三次握手
- 7.webpack打包流程
- 8.babel原理
- 9.tree shaking原理
- 10.输入一个url到页面显示整个过程
- 11.JS事件循环，宏任务、微任务
- 12.设计模式有没有了解过
- 13.项目webwork怎么用的
- 14.如何通过角度去计算弧度
- 15.如何判断一条线在一个平面内
- 16.写一个发布订阅
- 17.写一个前序遍历
- 18.写一个Promise.all

参考答案：

1. React Fiber

React Fiber 是 React 16 引入的重新实现的调度引擎，用于优化 UI 渲染性能。Fiber 的设计目的是提升 React 的可扩展性和性能，使得组件可以以非阻塞的方式进行渲染。它通过将渲染过程拆分为多个小的任务，来允许 React 在空闲时间中分配工作，从而避免长时间的 UI 阻塞。Fiber 支持优先级调度，使得高优先级的任务可以在低优先级任务之前完成。

2. HTTP/1.1、HTTP/2、HTTP/3

- HTTP/1.1: 是 HTTP/1.0 的改进版本，增加了持久连接、管道化、缓存控制等功能，但仍存在性能瓶颈，如头部冗余和队头阻塞。
- HTTP/2: 引入了二进制传输、流量控制和多路复用等功能。它通过多路复用允许同时在一个连接上处理多个请求，从而减少了延迟和头部冗余。
- HTTP/3: 基于 QUIC 协议，进一步减少了连接和传输延迟。QUIC 在用户数据报协议（UDP）上运行，改进了传输性能和安全性，同时也支持多路复用和加密。

3. HTTPS 和 HTTP 的区别，HTTP 是怎么加密的

- HTTP（HyperText Transfer Protocol）和 HTTPS（HyperText Transfer Protocol Secure）是两种用于传输数据的协议，但它们之间有几个关键区别，主要涉及安全性。

HTTP 与 HTTPS 的区别

1. 安全性:

- HTTP: 数据在传输过程中是明文的，易受到中间人攻击、窃听和篡改。
- HTTPS: 数据在传输过程中通过 SSL/TLS 协议加密，确保通信的安全性和数据的完整性。

2. 端口:

- HTTP: 默认使用端口 80。
- HTTPS: 默认使用端口 443。

3. 证书:

- HTTP: 不使用证书，通信不加密。
- HTTPS: 使用数字证书来验证服务器的身份。证书由受信任的证书颁发机构（CA）颁发，包含公钥和服务器身份的信息。

4. 性能:

- HTTP: 由于没有加密和解密的开销，性能略高。
- HTTPS: 加密和解密的过程会带来一定的性能开销，但现代计算机和服务器的性能足以处理这些开销，并且可以通过优化来减少影响。

HTTP 是怎么加密的

HTTP 本身不涉及加密，只有在 HTTPS 中，数据才会被加密。HTTPS 使用 SSL/TLS 协议来加密数据，过程如下：

1. SSL/TLS 握手:

- 客户端 发起与 服务器 的 SSL/TLS 握手请求。
- 服务器 发送其数字证书（包含公钥）给 客户端。
- 客户端 验证证书的有效性（通过受信任的证书颁发机构验证证书）。
- 客户端 生成一个对称密钥（预主密钥），用 服务器 的公钥加密后发送给 服务器。
- 服务器 使用其私钥解密出对称密钥，并与 客户端 使用相同的对称密钥进行加密通信。

2. 对称加密:

- 一旦握手完成，客户端 和 服务器 就开始使用对称密钥对数据进行加密和解密。
- 对称加密算法如 AES（Advanced Encryption Standard）用于加密实际的数据传输内容。

3. 数据完整性:

- 在 HTTPS 中，数据传输过程中还会使用消息认证码（MAC）来确保数据在传输过程中没有被篡改。

例子

• HTTP 请求:

```
1 GET /index.html HTTP/1.1
2 Host: example.com
```

• HTTPS 请求:

- 在握手完成后，实际数据内容会被加密，例如：

```
1 GET /index.html HTTP/1.1
2 Host: example.com
```

- 这些请求的内容在网络中是加密的，无法被直接读取或篡改。

4. OSI 七层网络模型

- 第 1 层: 物理层（Physical Layer） - 处理实际的硬件传输。
- 第 2 层: 数据链路层（Data Link Layer） - 处理节点间的链路建立和维护。
- 第 3 层: 网络层（Network Layer） - 处理数据包的路由选择。

- 第 4 层: 传输层 (Transport Layer) - 处理端到端的数据传输和错误检测。
- 第 5 层: 会话层 (Session Layer) - 处理会话管理和控制。
- 第 6 层: 表示层 (Presentation Layer) - 处理数据的表示和转换。
- 第 7 层: 应用层 (Application Layer) - 处理网络应用和用户接口。

5. TCP 和 UDP 区别

- TCP (Transmission Control Protocol):
 - 面向连接的，提供可靠的数据传输。
 - 有序传输，保证数据完整性。
 - 使用三次握手建立连接。
- UDP (User Datagram Protocol):
 - 无连接的，传输速度较快但不保证可靠性。
 - 数据包可能乱序或丢失。
 - 适用于实时应用，如视频流和在线游戏。

6. 三次握手

TCP 三次握手 是建立 TCP 连接的过程：

1. SYN: 客户端发送 SYN（同步）请求到服务器。
2. SYN-ACK: 服务器回复 SYN-ACK（同步-确认）。
3. ACK: 客户端发送 ACK（确认）以完成连接。

7. Webpack 打包流程

4. 初始化: 读取配置文件，生成 Compiler 对象。
5. 编译: 从入口文件开始递归解析模块，生成模块依赖图。
6. 构建: 根据依赖图生成相应的代码块。
7. 优化: 进行代码压缩、树摇（tree shaking）等优化处理。
8. 输出: 将最终的打包结果输出到指定目录。

8. Babel 原理

Babel 是一个 JavaScript 编译器，主要用于将现代 JavaScript（如 ES6+ 语法）转换为兼容性更强的旧版本 JavaScript（如 ES5），以便在不同的浏览器和环境中运行。它的核心原理可以分为以下几个步骤：

1. 解析 (Parsing)

- Babel 首先通过词法分析和语法分析，将代码解析成一种叫做**抽象语法树 (AST, Abstract Syntax Tree)** 的结构。AST 是一种以树形结构表示代码语法和结构的形式，这样方便后续的和生成步骤。

2. 转换 (Transformation)

- Babel 会根据配置的插件和预设 (presets) 对 AST 进行修改和转换。例如，如果使用了 ES6 的 `箭头函数`，Babel 会将其转换为等价的传统函数表达式。转换过程基于插件体系，允许用户自定义和扩展转换逻辑。

3. 生成 (Generation)

- 在完成 AST 转换后，Babel 将新的 AST 转回成标准的 JavaScript 代码。这个过程叫做代码生成，Babel 会尽可能保持原代码的风格，但会根据目标环境的需求将高级语法转换为低级的等效实现。

主要组成部分：

- **Parser (解析器)**：负责将源代码转换为 AST。常用的 parser 是 `@babel/parser`。
- **Transform (转换器)**：插件的核心部分，负责修改 AST。比如 `@babel/plugin-transform-arrow-functions` 插件可以将箭头函数转换为普通函数。
- **Generator (生成器)**：将 AST 转换回 JavaScript 代码，使用的是 `@babel/generator`。

Babel 的配置

Babel 的强大之处在于其插件系统和预设系统。预设是插件的集合，例如 `@babel/preset-env` 会根据你设定的目标环境自动选择适合的插件，把不同版本的 JavaScript 语法转换为目标环境支持的版本。

- **插件 (Plugins)**：每个插件负责将一种或几种特定的语法转换成等效的旧语法。例如，箭头函数、类、模块导入导出等高级特性都可以通过不同的插件来转换。
- **预设 (Presets)**：是插件的集合，提供一站式的转换方案，比如 `@babel/preset-env` 会根据目标浏览器的兼容性自动添加需要的插件。

工作流程

1. **安装和配置**：开发者配置 `.babelrc` 或 `babel.config.js`，指定需要的插件和预设。
2. **代码解析**：Babel 将源代码转换为 AST。
3. **AST 转换**：根据配置的插件修改 AST。
4. **代码生成**：最终生成转换后的代码，确保它可以在更多的环境中正常运行。

通过这个过程，Babel 让开发者可以使用最新的 JavaScript 语言特性，而不用担心浏览器的兼容性问题。

9. Tree Shaking 原理

Tree Shaking 是一种优化技术，用于删除未使用的代码。其原理如下：

1. 静态分析: 通过静态分析代码的模块依赖，识别出未被使用的代码。
2. 删除未使用的代码: 从最终的打包结果中移除这些未使用的代码。

10. 输入一个 URL 到页面显示的整个过程

3. DNS 查询: 将 URL 转换为 IP 地址。
4. TCP 握手: 与服务器建立 TCP 连接。
5. 发送 HTTP 请求: 向服务器发送请求。
6. 服务器处理请求: 服务器返回 HTTP 响应。
7. 浏览器渲染: 浏览器解析 HTML、CSS、JS，渲染页面。

11. JS 事件循环，宏任务、微任务

- 事件循环: JavaScript 执行环境的机制，用于处理异步操作。包括执行栈、宏任务队列和微任务队列。
- 宏任务: 如 `setTimeout`、`setInterval`、I/O 操作等。
- 微任务: 如 `Promise` 的 `then`、`catch`、`finally` 回调等。

12. 设计模式

设计模式 是软件开发中的最佳实践方案，用于解决常见问题。常见设计模式包括：

- 单例模式: 确保一个类只有一个实例。
- 工厂模式: 提供创建对象的接口。
- 观察者模式: 允许对象之间进行一对多的依赖关系。

13. 项目 WebWorker 怎么用的

WebWorker 允许在后台线程中运行 JavaScript，避免阻塞主线程。使用步骤：

1. 创建 Worker: 使用 `new Worker('worker.js')` 创建 Worker 实例。
2. 通信: 使用 `postMessage` 和 `onmessage` 进行主线程与 Worker 之间的通信。
3. 终止 Worker: 使用 `terminate()` 终止 Worker。
4. 如何通过角度计算弧度

角度转弧度: 使用公式 `radians = degrees * (Math.PI / 180)`，其中 `Math.PI` 为圆周率。

5. 如何判断一条线在一个平面内

判断线是否在平面内: 可以通过将线的两个点代入平面的方程，如果结果一致，则该线在平面内。

6. 写一个发布订阅

```
1 class PubSub {
2   constructor() {
3     this.subscribers = {};
4   }
5
6   subscribe(event, callback) {
7     if (!this.subscribers[event]) {
8       this.subscribers[event] = [];
9     }
10    this.subscribers[event].push(callback);
11  }
12
13  publish(event, data) {
14    if (this.subscribers[event]) {
15      this.subscribers[event].forEach(callback => callback(data));
16    }
17  }
18 }
```

7. 写一个前序遍历

```
1 function preOrderTraversal(root) {
2   if (!root) return [];
3   return [root.value].concat(preOrderTraversal(root.left),
4     preOrderTraversal(root.right));
5 }
```

8. 写一个 Promise.all

```
1 Promise.myAll = function(promises) {
2   return new Promise((resolve, reject) => {
3     let results = [];
4     let completed = 0;
5     // ...
```

```

6     promises.forEach((promise, index) => {
7         Promise.resolve(promise).then(value => {
8             results[index] = value;
9             completed++;
10            if (completed === promises.length) {
11                resolve(results);
12            }
13        }).catch(err => {
14            reject(err);
15        });
16    });
17 });
18 };

```

当然，我会尽量逐一回答你的问题。

1. 设计模式有没有了解过？

是的，设计模式是软件开发中常见问题的最佳实践解决方案。常见的设计模式包括：

- 创建型模式：单例模式、工厂模式、抽象工厂模式、建造者模式、原型模式。
- 结构型模式：适配器模式、代理模式、装饰模式、桥接模式、外观模式、享元模式、组合模式。
- 行为型模式：策略模式、模板方法模式、观察者模式、迭代器模式、责任链模式、命令模式、备忘录模式、状态模式、访问者模式、中介者模式、解释器模式。

2. 项目webwork怎么用的？

WebWork 是一个基于 Java 的 Web 应用框架，它是 Apache Struts 框架的前身之一。WebWork 提供了丰富的功能来处理 HTTP 请求、表单数据、验证、国际化等。使用 WebWork 通常涉及以下几个步骤：

1. 配置 WebWork：在 `web.xml` 中配置 WebWork 的过滤器（Filter）和 Servlet。
2. 创建 Action 类：处理用户请求的业务逻辑。
3. 配置 Action：在 `struts.xml` 中配置 Action 的路径、类名、结果页面等。
4. 创建 JSP 页面：用于显示数据和接收用户输入。
5. 验证和国际化：使用 WebWork 提供的验证框架和国际化支持。

14. 如何通过角度去计算弧度？

角度和弧度之间的转换公式是：

$$\text{弧度} = \text{角度} \times 180\pi$$

例如，要将 90 度转换为弧度，计算如下：

$$\text{弧度} = 90 \times 180\pi = 2\pi$$

15.如何判断一条线在一个平面内？

在三维空间中，一条线是否在一个平面内，可以通过以下步骤判断：

1. 表示线和平面：

- 线可以用两个点 $P1(x1,y1,z1)$ 和 $P2(x2,y2,z2)$ 表示。
- 平面可以用一般式 $Ax+By+Cz+D=0$ 表示。

2. 计算方向向量：

- 线的方向向量为 $d=(x2-x1,y2-y1,z2-z1)$ 。

3. 判断方向向量与平面的关系：

- 将方向向量代入平面方程，如果结果为 0，则线在平面内。

16. 写一个发布订阅

以下是一个简单的发布订阅（Publish-Subscribe）模式的 JavaScript 实现：

```
1 class EventEmitter {
2   constructor() {
3     this.events = {};
4   }
5
6   on(event, listener) {
7     if (!this.events[event]) {
8       this.events[event] = [];
9     }
10    this.events[event].push(listener);
11  }
12
13  emit(event, ...args) {
14    if (this.events[event]) {
15      this.events[event].forEach(listener => listener(...args));
16    }
17  }
18
19  off(event, listener) {
20    if (this.events[event]) {
21      this.events[event] = this.events[event].filter(l => l !==
22      listener);
23    }
24  }
25
26  // 使用示例
```

```

27 const emitter = new EventEmitter();
28 emitter.on('event', () => console.log('Listener 1'));
29 emitter.on('event', () => console.log('Listener 2'));
30 emitter.emit('event'); // 输出: Listener 1, Listener 2
31 emitter.off('event', () => console.log('Listener 1'));
32 emitter.emit('event'); // 输出: Listener 2

```

17. 写一个前序遍历

以下是一个二叉树的前序遍历（Pre-order Traversal）的 JavaScript 实现：

```

1 function preOrderTraversal(root) {
2     const result = [];
3     function traverse(node) {
4         if (node) {
5             result.push(node.val);
6             traverse(node.left);
7             traverse(node.right);
8         }
9     }
10    traverse(root);
11    return result;
12 }
13
14 // 示例二叉树
15 const tree = {
16     val: 1,
17     left: { val: 2, left: null, right: null },
18     right: { val: 3, left: null, right: null }
19 };
20
21 console.log(preOrderTraversal(tree)); // 输出: [1, 2, 3]

```

18. 写一个 Promise.all

以下是一个模拟 `Promise.all` 的 JavaScript 实现：

```

1 function myPromiseAll(promises) {
2     return new Promise((resolve, reject) => {
3         const results = [];
4         let count = 0;
5

```

```
6     promises.forEach((promise, index) => {
7         Promise.resolve(promise).then(value => {
8             results[index] = value;
9             count++;
10            if (count === promises.length) {
11                resolve(results);
12            }
13        }).catch(reject);
14    });
15 });
16 }
17
18 // 使用示例
19 myPromiseAll([
20     Promise.resolve(1),
21     Promise.resolve(2),
22     Promise.resolve(3)
23 ]).then(values => console.log(values)); // 输出: [1, 2, 3]
```